

VitalsPatch Capstone Report

A Wearable Multi-Patient Health Monitoring System

Eeshani Shilamkar, Arnav Mohan, Maya Desai, Bobby Taing, Anushka Misra, Nate Snyder
EE 475 Embedded Systems Capstone - Spring 2026

Sponsor of Project: University of Washington Dept. of Electrical and Computer Engineering
Dept. of Electrical and Computer Engineering, Box 352500 University of Washington
Seattle, Washington 98195 U.S.A.

June 8, 2026

Table of Contents

Table of Contents	2
Team, Roles, and Responsibilities	4
Product Requirements Document	5
Product Need	5
Where Will It Be Used?	5
Who Will Use It?	5
How Will It Be Used?	5
What Will It Be Used With?	6
Value Proposition	6
Realistic Constraints and Engineering Standards	6
1. Technical Constraints	6
2. Non-Technical Constraints	8
3. Engineering Standards	9
System Requirements Document	10
1. Functional Requirements	12
2. Size, Weight, Cost Requirements	12
3. Power Requirements	12
4. Communication and Interface Requirements	13
5. Software and Firmware Requirements	13
6. Data Storage, Format, and Security Requirements	13
7. Precision and Accuracy Requirements	14
8. User Interface Requirements	14
9. Test and Validation Requirements	15
10. Safety Requirements	15
11. Patient and Clinical Requirements	15
Project Schedule	16
Project Resources	17
Outline of Experiments	18
Phase 1: Sensor Bring-Up and Basic Transmission	18
Objective	18
Procedure	18
Metrics Evaluated	18
Phase 2: STM32 Connectivity and Centralized Processing	18
Objective	18
Procedure	18
Metrics Evaluated	19
Phase 3: Alert Logic Migration, Calibration Refinement, and Full Integration	19
Objective	19
Procedure	19

Metrics Evaluated	19
Trial Designs	20
Phase 1 Design: Baseline Sensor and Transmission Prototype	20
Phase 2 Design: STM32-Centered Processing Architecture	20
Phase 3 Design: Distributed Clinical Monitoring Architecture	20
Final System Architecture	20
Sensor Node Hardware	21
Experimental Outcomes	21
Phase 1 Outcomes: Sensor Bring-Up and Basic Transmission	22
Phase 2 Outcomes: STM32 Connectivity and Processing	22
Phase 3 Outcomes: Full System Verification	22
Heart Rate Accuracy	22
SpO ₂ Accuracy	23
Temperature Accuracy	23
Fall Detection	24
End-to-End Latency and Dashboard Verification	24
Impact and Consequences	24
Conclusions and Recommendations	25
References	27
Acknowledgements and Intellectual Property	29

Team, Roles, and Responsibilities

The project is divided across six members so that the sensor, processing, communication, and interface subsystems can be developed in parallel. As development progressed, responsibilities evolved to address technical challenges, integration issues, and schedule delays.

Team Member	Role and Responsibility
Eeshani Shilamkar	User-Interface design and testing, sensor calibration and testing, debugging, and data analysis algorithm. Project management.
Arnav Mohan	Sensor integration, data analysis algorithm, debugging, and wireless communication.
Maya Desai	Data analysis algorithm, system integration, debugging, and sensor calibration and testing
Bobby Taing	Wireless communication and system integration.
Nate Snyder	Assigned responsibility for wearable development support, sensor calibration support, and prototype testing activities.
Anushka Misra	Assigned responsibility for wearable hardware assembly, soldering, wearable testing, and sensor side acquisition support.

Table 1. A table connecting team members to their roles in the project.

Throughout the project, the team encountered challenges related to completion of the wearable hardware subsystem. Wearable assembly, soldering, and validation tasks experienced significant delays, which impacted the team's ability to complete the final wearable prototype within the project timeline. Although progress updates from respective members indicated that wearable development was underway, notification that the wearable assembly could not be completed was received late in the development cycle (two days before the phase 3 demo), leaving limited time to reassign the work before the major project milestones.

While the core system functionality was completed and demonstrated, these delays within the wearable hardware workstream prevented completion of the final wearable packaging and validation activities. The team prioritized completion of the end-to-end monitoring pipeline, including physiological sensing, wireless communication, ESP processing, alert generation, and Raspberry Pi dashboard visualization, ensuring that the primary technical objectives of the project were successfully achieved despite limitations in available time and personnel resources.

Product Requirements Document

Product Need

Continuous monitoring of patient vital signs is critical in hospitals, outpatient clinics, nursing homes, and assisted-living facilities, where rapid response to a fall or sudden change in vital signs can directly affect patient outcomes. Existing wearable monitoring devices typically focus on a single physiological measurement, requiring healthcare providers to combine multiple devices to obtain a complete picture of a patient's condition. This increases complexity, cost, and the likelihood that important changes in patient health may go unnoticed. VitalsPatch addresses this need by providing a single wearable device capable of continuously monitoring multiple vital signs while automatically alerting healthcare providers when abnormal conditions are detected.

Where Will It Be Used?

VitalsPatch is intended for use in outpatient clinics, rehabilitation centers, nursing homes, assisted-living facilities, and other healthcare environments where patients would benefit from continuous monitoring. The system is particularly valuable in settings where a limited number of healthcare professionals are responsible for monitoring multiple patients simultaneously.

Who Will Use It?

The primary users of VitalsPatch are healthcare professionals, including nurses, caregivers, clinical staff, patient monitoring personnel, and the patients themselves. Patients wear the wireless device, while healthcare providers interact with the monitoring dashboard to view patient status, identify trends, and respond to alerts.

How Will It Be Used?

Each patient wears a VitalsPatch device that continuously collects heart rate, blood oxygen saturation (or SpO₂), body temperature, and motion data. The system automatically analyzes the collected information and detects abnormal vital signs or fall events. Healthcare providers monitor patient status through a centralized dashboard that displays live patient data and health alerts when intervention may be required.

What Will It Be Used With?

VitalsPatch is designed to operate with a wearable sensor module with an ESP32-based processing unit, wireless communication BLE modules on the STM32, and a Raspberry Pi UI dashboard. The system works alongside existing healthcare workflows by providing healthcare professionals with a centralized platform for monitoring multiple patients simultaneously.

Value Proposition

Several existing products address portions of this problem, including FloPatch for blood-flow monitoring, Aktiia for cuffless blood-pressure monitoring, and the EvieMed Ring for pulse oximetry. However, these products focus on individual measurements and do not provide centralized multi-patient monitoring. VitalsPatch combines heart rate, SpO₂, temperature, and fall detection into a single wearable device while providing a centralized dashboard that allows one healthcare professional to monitor multiple patients from a single interface.

Realistic Constraints and Engineering Standards

The VitalsPatch system and pipeline were designed to satisfy the requirements identified in the PRD while operating within practical engineering, safety, and project constraints. As a wearable patient-monitoring device, the system was required to continuously acquire physiological data, communicate wirelessly, process sensor measurements in real time, and generate alerts when abnormal conditions were detected. These requirements introduced limitations associated mainly with sensor capabilities and communication hardware. Additional constraints arose from power consumption, development resources, and healthcare-related safety requirements.

To ensure the design remained both technically feasible and aligned with its intended clinical use, the project was guided by a combination of technical constraints, non-technical constraints, and relevant engineering standards. Some constraints were determined from hardware specifications and accepted clinical thresholds, while others reflected design decisions made to accommodate project scope, available resources, and realistic development timelines.

1. Technical Constraints

Table 1 lists the technical constraints of the system. Constraints marked rigid represent physical hardware limits or published clinical safety thresholds that cannot be changed without replacing a component or disregarding patient safety. Negotiable constraints represent tunable parameters that were set based on reasonable parameters or requirements but could be adjusted.

Constraint	Value / Limit	Rigid or Negotiable	Design Decision Driven
MAX30102 SpO ₂ range	70 – 100% [9]	Rigid (hardware limit)	Sensor selected because datasheet range covers full clinical SpO ₂ spectrum [1], [2]
MAX30102 HR range	40 – 200 BPM [9]	Rigid (hardware limit)	Validated against Apple Watch and manual CNA counts as reference baseline [3]

SpO ₂ alert threshold	< 90% triggers alert [9]	Rigid (clinical standard)	Threshold set per published hypoxemia guidelines; not adjustable by end user [3]
HR alert – high	> 120 BPM [3]	Negotiable	Based on standard tachycardia threshold; could be tuned per patient profile [3]
HR alert – low	< 60 BPM [3]	Negotiable	Based on standard bradycardia threshold; could be tuned per patient profile [3]
Core temp – fever alert	> 100.4°F (38.0°C) [4]	Rigid (clinical standard)	CDC-defined fever threshold; non-negotiable for patient safety [4]
Core temp – hypothermia alert	< 95.0°F (35.0°C) [4]	Rigid (clinical standard)	Standard clinical hypothermia threshold [4]
Finger temp – high alert	> 90°F (32.2°C) sustained [5]	Negotiable	Peripheral threshold used in demo due to finger placement; would be replaced by core thresholds in a patch form factor
Finger temp – low alert	< 80°F (26.7°C) sustained [5]	Negotiable	Peripheral threshold used in demo due to finger placement; would be replaced by core thresholds in a patch form factor
Fall detection trigger	fallDetected == true via MPU-6050	Rigid	IMU threshold tuned to detect rapid downward hand motion in the handheld prototype
I ² C bus	Shared bus: MAX30102 + MPU-6050 on ESP32-C3 [11]	Rigid (hardware)	Both sensors are I ² C-only; shared bus was the only feasible wiring on the ESP32-C3
BLE service UUID	FFE0 / FFE1 (HM-19 UART transparent) [12]	Rigid (module constraint)	HM-19 module only exposes this service; drove choice of BLE stack on ESP32-C3
Estimated peak current	~150–200 mA (datasheet estimate) [9], [10], [11]	Rigid	LiPo not soldered in final prototype

Target battery	3.7V LiPo	Rigid	Selected for energy density and wearable form factor
Dashboard latency	< 20 s end-to-end (target)	Negotiable	Informal target based on acceptable clinical response time; not formally measured

Table 2. Technical constraints and their impact on design decisions.

2. Non-Technical Constraints

While the technical specifications of the VitalsPatch system define its sensing, communication, and performance capabilities, several non-technical constraints were considered in shaping the final design. The project was completed within a ten-week academic quarter, which significantly limited our technology selection and development priorities. Because the team already had experience with Arduino-based development, the ESP32-C3 was selected as the primary sensing and Bluetooth Low Energy (BLE) platform instead of relying exclusively on the STM32F4. Although the STM32 offered greater processing capability, development complexity and schedule risk made the ESP32-C3 a more practical choice. The wearable form-factor requirement further reinforced this decision, as the STM32 development board was too large to integrate directly into a body-worn patch. Instead, the ESP32-C3 served as the wearable sensing node while the STM32 was retained as initially a processing module, and later a communication bridge.

Team expertise and hardware availability also affected implementation decisions. The development team's familiarity with Arduino tools reduced firmware development time and enabled parallel progress on sensor integration and wireless communication. Several components were selected based on laboratory availability and procurement constraints rather than optimal commercial design choices. For example, the HM-19 Bluetooth module was chosen because it was readily available [12], leading to the use of the module's UART communication architecture. Similarly, commercially available breakout boards for the MAX30102 pulse oximeter and MPU-6050 inertial measurement unit were used instead of designing custom PCBs for the sensor circuitry. Budget limitations favored the use of commercial modules and existing hardware resources over a fully integrated custom PCB. Safety considerations also influenced the design. Because the device is intended to contact human skin, all sensing electronics operate from a low-voltage 3.3V [9], [10], [11] supply and contain no mains-connected circuitry. Finally, patient data privacy was recognized as an important future requirement. Although the academic prototype transmits data without encryption and stores dashboard information locally, a production version would require secure transmission, protected storage, user authentication, and compliance with healthcare privacy regulations.

3. Engineering Standards

Engineering standards were also considered throughout the design process because they directly influenced key architectural decisions. Rather than simply selecting components that happen to conform to industry standards, the design team evaluated how specific standards affected communication methods, power consumption, safety requirements, interoperability, and future deployment considerations. The Bluetooth communication system was designed around the requirements of IEEE 802.15.1 [6], which governs Bluetooth physical and link-layer operation. Compliance with this standard supported the decision to use BLE rather than Wi-Fi, since BLE offered significantly lower power consumption while still providing adequate communication range for a single-room monitoring environment. The sensor subsystem was similarly constrained by the Inter-Integrated Circuit (I²C) specification, since both the MAX30102 and MPU-6050 communicate exclusively through I²C. The ability to share a common two-wire bus using unique device addresses simplified wiring and reduced the number of required microcontroller pins.

Medical-device standards were considered primarily as future deployment requirements rather than immediate compliance obligations. IEC 60601-1 [7], which governs electrical safety for medical electrical equipment, influenced the decision to operate the wearable entirely from a low-voltage battery-powered supply with no direct mains connection. Although formal compliance testing was beyond the scope of the project, the standard guided some safety design choices. Similarly, ISO 80601-2-61, which historically defined performance requirements for pulse oximeters, served as a benchmark for evaluating the MAX30102 sensor. Although this standard has since been withdrawn, the sensor's manufacturer-specified accuracy falls within the limits that were required under the standard at the time of its publication. Regulatory considerations also influenced wireless hardware selection. The ESP32-C3 module was selected partly because it is FCC-certified for operation in the 2.4 GHz ISM band, reducing future regulatory requirements. Healthcare privacy regulations such as HIPAA were also considered, although encryption and other security measures were not implemented in the academic prototype.

Not all standards were applied to the same degree. Standards governing communication protocols, such as Bluetooth and I²C, were followed directly because they were necessary for the entire system operation. Medical safety, performance, and privacy standards were considered during the design process and influenced component selection and system architecture, but full compliance was unmet because the project was developed as an academic capstone project rather than a commercial medical device. Even so, consideration of these standards helped ensure that the design decisions made during development remain compatible with a future pathway toward real clinical deployment and regulatory approval.

System Requirements Document

The VitalsPatch system consists of three primary modules: a sensor/data acquisition and processing module, a wireless communication module, and a Raspberry Pi user interface module. Together, these modules provide continuous physiological monitoring, abnormal event detection, and multi-patient visualization.

The sensor and processing subsystem is implemented on an ESP32C3 microcontroller, which collects physiological and motion data, performs threshold-based event detection, and packages processed measurements for transmission. Wireless communication between the wearable device and the monitoring station is provided through HM-19 Bluetooth Low Energy modules hosted on the STM32. This STM32 microcontroller acts as a communication path, receiving multiple streams of wireless data, and forwarding it through a wired serial connection to the Raspberry Pi. The Raspberry Pi hosts the monitoring dashboard, displays patient trends, maintains event logs, and presents prioritized alerts. Figure 1 demonstrates this entire integrated system for two patients as a UML chart.

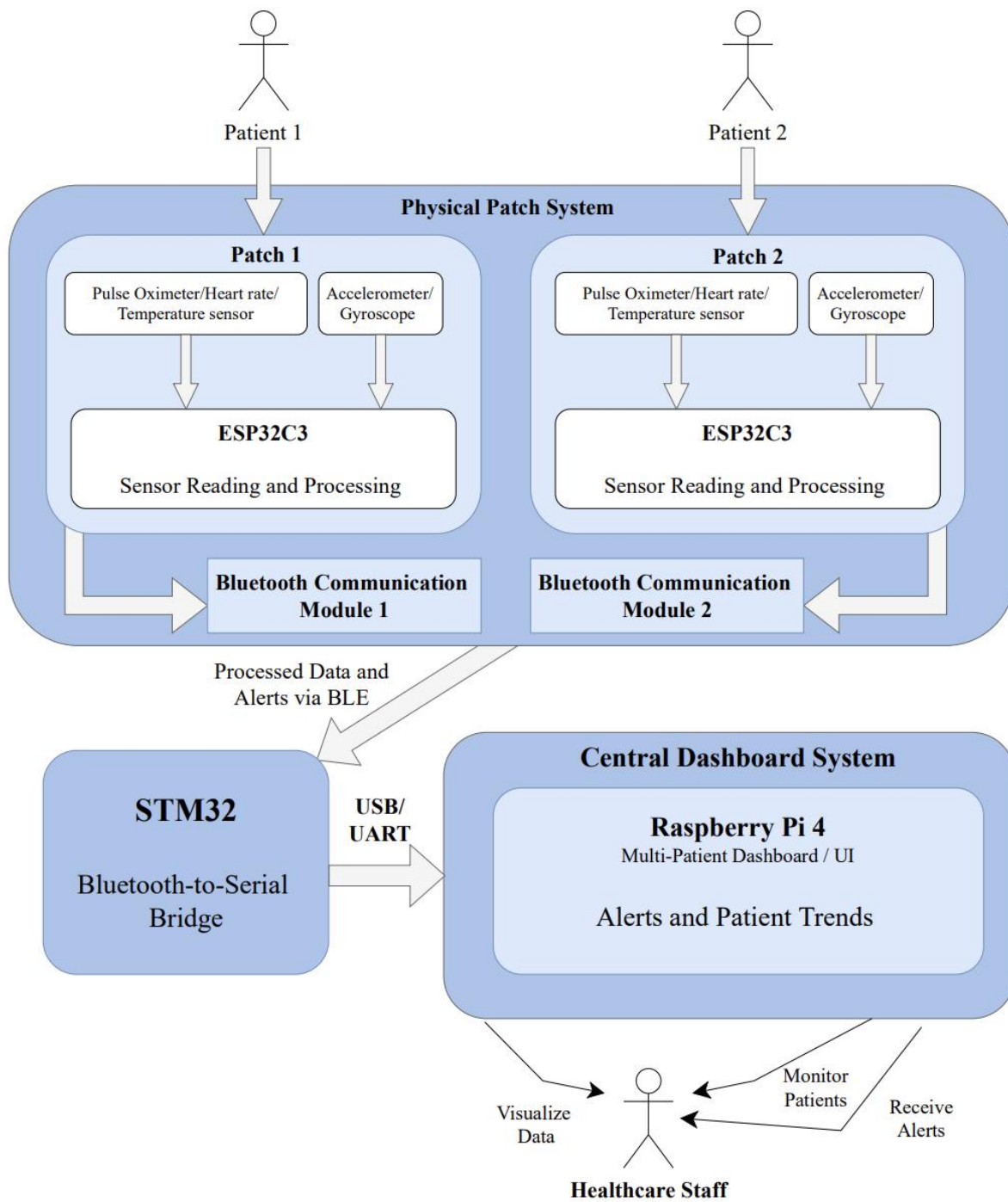


Fig. 1 - UML chart of the entire integrated system for two patients.

1. Functional Requirements

Requirement Number	Requirement
1.1	The system should continuously acquire heart rate, SpO ₂ , temperature, accelerometer, and gyroscope measurements from each connected patient device.
1.2	The system should detect abnormal heart rate (high or low heart rate), abnormal temperature (high or low temperature), low SpO ₂ , and fall events and generate the corresponding 6 alert codes.
1.3	The system should support simultaneous data acquisition, processing, and display of data from at least two patients.

Table 3. Functional requirements of a VitalsPatch

2. Size, Weight, Cost Requirements

Requirement Number	Requirement
2.1	The wearable patch shall be compact, wireless, and suitable for placement on the upper back, arm, or shoulder.
2.2	The prototype of the system should be constructed using commercially available off-the-shelf components.

Table 4. Size, Weight, and Cost requirements of a VitalsPatch

3. Power Requirements

Requirement Number	Requirement
3.1	The ESP32-C3 data acquisition and processing module should operate with a portable battery power source between 3.3 V to 5 V. [11]
3.2	The Raspberry Pi UI module should operate from an external, consistent power supply of 5.1V.

Table 5. Power requirements of a VitalsPatch

4. Communication and Interface Requirements

Requirement Number	Requirement
4.1	The ESP32-C3 data acquisition and processing module should transmit processed patient data using Bluetooth Low Energy communication.
4.2	The STM32 wireless communication module should receive Bluetooth data with a BLE module, and forward it to the Raspberry Pi through a wired serial connection at a consistent sampling rate. Each BLE module should only be connected to one ESP32C3 patient data acquisition module.
4.3	The entire integrated system should support wireless bluetooth communication from at least two independent patient devices at a consistent sampling rate.
4.4	The packets of data sent between all three modules should have a patient identifier, an array of 6 alert codes (with the triggered alert being set to 1), and 3 heart rate, temperature, and SpO ₂ data points. It should be in this example format: Patient:1;HR:[75,76,74];SPO2:[98,99,97];TEMP:[98.4,98.5,98.3];ERR:[0,0,0,0,0,0]

Table 6. Communication and Interface requirements of a VitalsPatch

5. Software and Firmware Requirements

Requirement Number	Requirement
5.1	The ESP32-C3 data processing and acquisition module should perform sensor data acquisition and processing, event / alert detection, and the wireless communication data packet generation.
5.2	The STM32 module should manage the incoming wireless bluetooth data from the ESP32s and serial data forwarding to the Raspberry Pi.
5.3	The Raspberry Pi software should sort the incoming patient data by the patient, and update that patient's respective dashboard in near real time.

Table 7. Software and Firmware requirements of a VitalsPatch

6. Data Storage, Format, and Security Requirements

Requirement Number	Requirement
6.1	All Patient data shall remain separated throughout acquisition, transmission, processing, and display. At the display stage, the data should be kept on premises (not on a third-party server) and should remain strictly in compliance with patient confidentiality and HIPAA laws.

Table 8. Data Storage, Format, and Security requirements of a VitalsPatch

7. Precision and Accuracy Requirements

Requirement Number	Requirement
7.1	Heart rate measurements shall be validated against a commercial reference device. The measurements should have an accuracy of between +/- 4 beats per minute to the reference device. This standard is consistent with the standards of the Certified Nursing Assistant exam.
7.2	Temperature measurements shall be validated against a reference thermometer.
7.3	Fall detection and alert generation shall be verified using simulated fall events and predefined abnormal physiological test cases.

Table 9. Precision and Accuracy requirements of a VitalsPatch

8. User Interface Requirements

Requirement Number	Requirement
8.1	The dashboard shall display near real-time graphs for heart rate, temperature, and SpO ₂ measurements.
8.2	The dashboard shall clearly distinguish between monitored patients.
8.3	The dashboard shall maintain patient-specific event logs and update dynamically as new data is received.
8.4	Alerts shall be displayed with higher visual priority than physiological trend data. When an active alert is present, the dashboard shall display the alert in a dedicated banner at the top of the user interface. The alert banner shall remain visible regardless of which patient's dashboard is currently being viewed.

Table 10. User Interface requirements of a VitalsPatch

9. Test and Validation Requirements

Requirement Number	Requirement
9.1	Multi-patient monitoring functionality can be verified using at least two simultaneous patient devices. Although, testing with more than two devices is preferred.
9.2	Sensor functionality shall be verified independently before full system integration. See requirement 7.1 for accuracy requirements.
9.3	End-to-end communication shall be verified by transmitting physiological data from the ESP32-C3 through the STM32 communication gateway and displaying the received data on the Raspberry Pi dashboard.

Table 11. Test and Validation requirements of a VitalsPatch

10. Safety Requirements

Requirement Number	Requirement
10.1	The system should operate using low-voltage electronics appropriate for wearable health-monitoring applications with a maximum voltage source of 3.7 V on the patient.
10.2	The wearable patch should be waterproof and sealed on all sides to ensure a dry environment around the electronics.

Table 12. Safety requirements of a VitalsPatch

11. Patient and Clinical Requirements

Requirement Number	Requirement
11.1	The patients used to test the device should be 55 years and older on the basis that this is the biggest use-case of the product.

Table 13. Patient and Clinical requirements of a VitalsPatch

Project Schedule

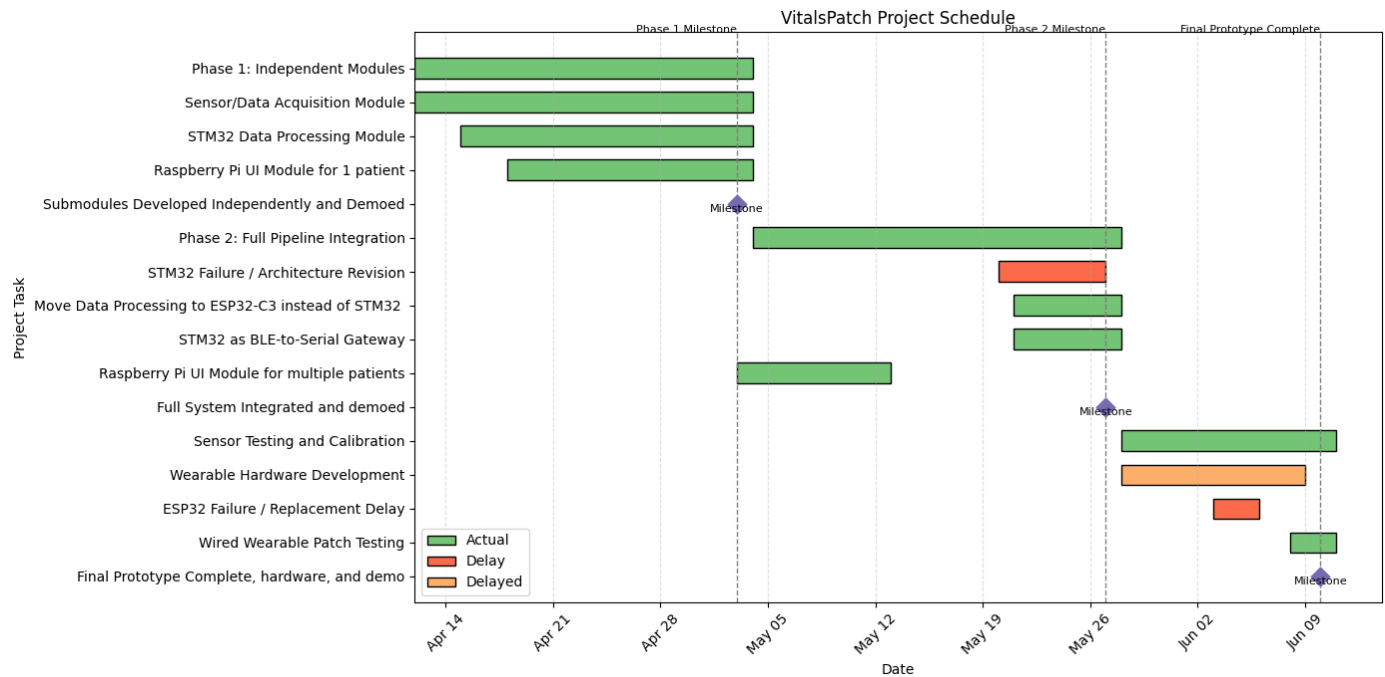


Fig. 2 - Gantt Chart displaying the progress of tasks, delays, and major milestones. “Actual” indicates the task was completed. “Delay” indicates the task was temporarily delayed, but was resolved. “Delayed” indicates the task was unable to be completed by the final demo deadline (June 8, 2026).

Project Resources

The team made use of the UW ECE undergraduate laboratory for hardware assembly, sensor bring-up, integration testing, and demo preparation. The laboratory provided access to bench power supplies, digital multimeters, and oscilloscopes used during sensor calibration and communication debugging. The Raspberry Pi dashboard was hosted on a lab workstation during all demo sessions.

The hardware components used in the project are listed in Table 14 below. The STM32 development board and Raspberry Pi 4 were provided by the UW ECE department at no cost to the team. All remaining components were ordered commercially, with the project staying within a modest budget of approximately \$40.64 in out-of-pocket component costs. A 3.7V LiPo battery was budgeted for the wearable power subsystem but was not installed in the final prototype due to the incomplete wearable assembly.

Component	Purpose	Cost
Seeed Studio XIAO ESP32-C3 [11]	Wearable MCU, BLE, sensor host	~\$6.66 each (\$19.99 for 3)
MAX30102 breakout board [9]	Heart rate and SpO ₂ sensing	~\$7.00 each (\$13.99 for 2)
MPU-6500 breakout board [10]	Fall detection via accelerometer/gyroscope	~\$5.00 each (\$14.99 for 3)
HM-19 BLE module (×2) [12]	UART-transparent BLE link (TX and RX)	~\$11.99 each
STM32 Nucleo board [13]	BLE receive bridge and serial forwarding	UW ECE Lab
Raspberry Pi 4 [14]	Dashboard host	UW ECE Lab
3.7V LiPo battery (planned)	Wearable power supply	~\$9.99 each (2 units)
Breadboard, jumper wires	Prototyping	UW ECE Lab

Table 14. Project hardware components and Cost

Software tools used throughout the project included the Arduino IDE for ESP32-C3 firmware development, the NimBLE-Arduino and SparkFun MAX3010x libraries [15] for BLE and sensor drivers respectively, STM32CubeIDE for the STM32 receive firmware, and Python and Node.js for the Raspberry Pi dashboard application. Version control was managed through GitHub, and all team documentation was produced collaboratively using Google Docs and Microsoft Word.

Outline of Experiments

The central R&D question guiding the VitalsPatch project was: can a single low-cost embedded platform reliably acquire heart rate, SpO₂, body temperature, and fall events, transmit them wirelessly to a centralized dashboard, and generate threshold-based physiological alerts in near real time? Development proceeded iteratively across three development phases, with each phase building on validated subsystems from the prior one.

Phase 1: Sensor Bring-Up and Basic Transmission

Objective

The objective of Phase 1 was to verify that the physiological and motion sensors could communicate with the ESP32-C3, produce valid measurements, and transmit data wirelessly through the BLE communication link.

Procedure

The MAX30102 pulse oximeter and MPU-6500 inertial measurement unit were connected to the ESP32-C3 through a shared I²C bus. Heart rate and SpO₂ measurements were generated using the Maxim Integrated algorithm provided by the SparkFun MAX3010x library. Temperature was obtained from the MAX30102 internal temperature sensor and converted to Fahrenheit without calibration correction. Three sensor readings were collected at one-second intervals and transmitted as a BLE packet every three seconds. No alert logic was enabled during this phase; the alert field was fixed to a placeholder value to verify packet formatting and communication.

Metrics Evaluated

- Successful sensor initialization and communication
- Valid heart rate and SpO₂ outputs
- Successful BLE packet transmission and reception
- Correct packet formatting and parsing

Phase 2: STM32 Connectivity and Centralized Processing

Objective

The objective of Phase 2 was to evaluate multi-patient communication, centralized alert processing, and fall-detection functionality using the STM32F4 as the primary processing node.

Procedure

Two HM-19 BLE modules were connected to separate UART channels on the STM32F4 to support two independent patient nodes. Communication reliability was verified through test transmissions received from each channel. Sensor measurements received from the wearable nodes were processed on the STM32 using circular data buffers and threshold-based alert logic. A four-state fall-detection algorithm was implemented and evaluated using simulated

motion sequences representing free-fall, impact, orientation change, and post-fall stillness. Processed data and alert information were forwarded to the Raspberry Pi dashboard through a serial connection.

Metrics Evaluated

- Independent communication on both BLE links
- Correct operation of alert-generation logic
- Fall-detection state-machine functionality
- Successful forwarding of processed data to the dashboard

Phase 3: Alert Logic Migration, Calibration Refinement, and Full Integration

Objective

The objective of Phase 3 was to implement the final system architecture by moving alert processing onto each wearable node, refining temperature calibration, and validating complete end-to-end system operation.

Procedure

Alert generation and fall detection were migrated from the STM32F4 to the ESP32-C3 wearable node. Sensor measurements were collected using an expanded averaging window to improve stability and reduce the impact of invalid readings. Temperature measurements were calibrated using an experimentally determined correction factor before conversion to Fahrenheit. Clinical alert thresholds for heart rate, SpO₂, temperature, and fall events were implemented and encoded into transmitted packets. Dual-patient support was incorporated through configurable patient identifiers and dedicated BLE connections. The complete sensing, communication, processing, and dashboard pipeline was then evaluated under normal and alert-generating conditions.

Metrics Evaluated

- Heart-rate accuracy
- SpO₂ accuracy
- Temperature accuracy
- Fall-detection performance
- BLE communication reliability
- End-to-end latency
- Dashboard alert verification

The results obtained from each phase are presented in the Experimental Outcomes section and are compared against the requirements defined in the System Requirements Document.

Trial Designs

The VitalsPatch system was developed across three design phases, with each phase representing a distinct firmware architecture. Rather than implementing the final system immediately, the design evolved iteratively as limitations were identified and addressed during development and integration. Each phase introduced modifications to the sensing, communication, processing, and alert-generation subsystems while maintaining the overall goal of continuous multi-patient vital-sign monitoring.

Phase 1 Design: Baseline Sensor and Transmission Prototype

The Phase 1 design focused on establishing a minimum viable sensing and communication pipeline. The ESP32-C3 acquired heart rate, SpO₂, temperature, and motion data from the MAX30102 and MPU-6500 sensors and transmitted packets over BLE at three-second intervals. No alert-generation or fall-detection functionality was included. This design served as a proof of concept for sensor integration, signal acquisition, and wireless transmission.

Phase 2 Design: STM32-Centered Processing Architecture

The Phase 2 design introduced support for multiple patients and centralized data processing. Two HM-19 BLE modules were connected to an STM32F4 microcontroller, allowing simultaneous reception of data from multiple wearable nodes. Alert generation and fall detection were implemented on the STM32 using buffered sensor data and threshold-based analysis. This architecture simplified firmware development but concentrated all computation on a single processing node.

Phase 3 Design: Distributed Clinical Monitoring Architecture

The Phase 3 design migrated alert generation and fall detection from the STM32F4 to each ESP32-C3 wearable node. This distributed-processing approach reduced scalability limitations by allowing each patient device to perform its own analysis before transmission. Additional refinements included temperature calibration, improved averaging of physiological measurements, configurable patient identification, and support for dual-patient monitoring. The STM32F4 was reduced to a communication bridge that forwards received data to the Raspberry Pi dashboard.

Final System Architecture

The final VitalsPatch architecture (as shown in the figure 3 block diagram) consists of a wearable sensor node based on the Seeed Studio XIAO ESP32-C3, a BLE communication link using HM-19 UART-transparent modules, an STM32F4 receive bridge, and a Raspberry Pi dashboard. The ESP32-C3 operates as a BLE central device, connecting to a designated HM-19 module identified by MAC address. The HM-19 converts BLE traffic to UART [12], allowing the STM32F4 to receive and forward packets to the Raspberry Pi through a serial connection.

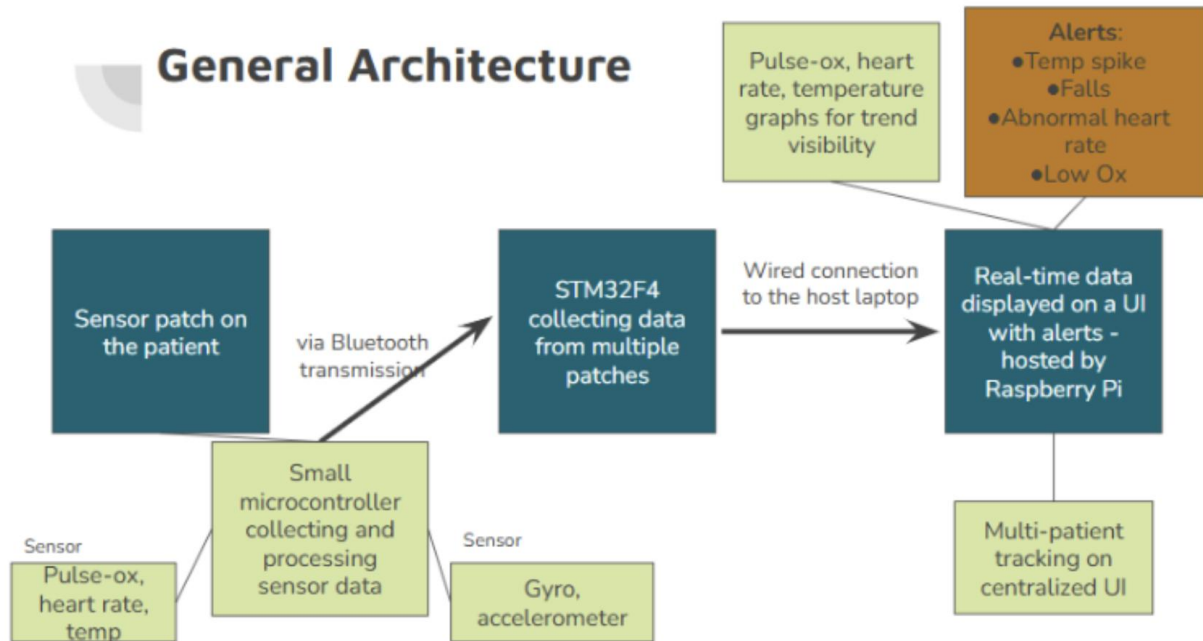


Fig. 3 - Block diagram showing the final architecture of the VitalsPatch.

Sensor Node Hardware

Both the MAX30102 and MPU-6500 share the I²C bus on GPIO 6 (SDA) and GPIO 7 (SCL) at 400 kHz [9], [10]. Table 15 summarizes pin assignments and sensor addresses.

Signal	ESP32-C3 Pin	Sensor	I ² C Address
I ² C SDA	GPIO 6	MAX30102, MPU-6500	0x57 [9], 0x68 [10]
I ² C SCL	GPIO 7	MAX30102, MPU-6500	0x57 [9], 0x68 [10]
UART TX	GPIO 21	HM-19 RX	-
UART RX	GPIO 20	HM-19 TX	-

Table 15. ESP32-C3 Sensor and Communication Interface Pin Mapping

Experimental Outcomes

System verification was performed by testing each subsystem independently across the three development phases and then validating the integrated pipeline end-to-end. Results are

compared against the requirements established in the SRD. Table 5 summarizes the verification status of all major requirements.

Phase 1 Outcomes: Sensor Bring-Up and Basic Transmission

Phase 1 testing verified successful communication between the ESP32-C3 and both sensing devices. The MAX30102 responded correctly at I²C address 0x57, while the MPU-6500 responded at address 0x68 and returned the expected WHO_AM_I value [10]. Under stable fingertip contact, the Maxim heart-rate and SpO₂ algorithm consistently produced valid outputs, indicating successful sensor acquisition and signal processing. BLE communication testing confirmed that packets generated by the ESP32-C3 were successfully transmitted, received, and parsed by the Raspberry Pi receiver using the three-second reporting cycle. The hardcoded ERR field of [0,0,0,0,0] was received correctly, confirming that packet formatting remained consistent throughout the communication pipeline.

Temperature measurements collected during Phase 1 exhibited a consistent offset below thermometer reference values. This observation motivated the calibration correction introduced during Phase 3 development.

Phase 2 Outcomes: STM32 Connectivity and Processing

Phase 2 testing verified reliable dual-patient communication and centralized processing functionality. Independent communication links were successfully established between two ESP32-C3 nodes and their corresponding HM-19 receivers connected to USART2 and USART3 on the STM32F4. LED-based communication tests confirmed that both UART-transparent BLE channels operated independently without observable cross-talk.

Alert-processing functionality implemented on the STM32F4 correctly applied threshold-based alert logic to simulated physiological data. The four-state fall-detection state machine progressed through the expected transitions when supplied with representative accelerometer and gyroscope inputs. Processed output packets generated by the STM32F4 were successfully forwarded to and parsed by the Raspberry Pi dashboard.

Integration testing identified a scalability limitation within the centralized processing architecture. Because all alert generation and fall-detection calculations were performed on a single STM32F4, computational demand would increase proportionally with the number of monitored patients. Additionally, midway through our full-system integration and testing, our STM32F4 had a hardware failure that pushed us in the direction of moving the alert generation and fall detection onto the ESP32-C3s. This finding motivated the transition to distributed processing in Phase 3.

Phase 3 Outcomes: Full System Verification

Heart Rate Accuracy

Heart-rate accuracy was evaluated by comparing VitalsPatch measurements against simultaneous Apple Watch readings and manual pulse counts obtained over 60-second intervals. Testing was conducted on participants with both light and dark skin tones, and no consistent performance differences were observed across the test samples collected from participants with different skin tones. The device computes heart rate using a rolling average of

the eight most recently detected beats and records three heart-rate samples over each 20-second reporting interval, approximately 6.7 seconds apart. This multi-stage averaging approach produced stable measurements during periods of consistent fingertip contact while reducing the influence of short-term beat-to-beat variability. Motion artifacts reduced measurement accuracy, which is consistent with the known limitations of reflective photoplethysmography sensors. Additionally, invalid-beat filtering excluded physiologically implausible measurements before averaging, preventing clearly erroneous readings from affecting the reported heart-rate values.

Trial	Condition	VitalsPatch (BPM)	Reference (BPM)	Error (BPM)
1	Light skin tone	75	73	+2
2	darker skin tone	82	86	-4

Table 16. Heart-rate accuracy results

SpO₂ Accuracy

SpO₂ measurements were compared against a reference pulse oximeter and Apple Watch readings during periods of stable fingertip contact. The Maxim algorithm consistently generated valid readings under low-motion conditions. Invalid outputs were excluded from averaging to prevent distortion of reported values. Testing was limited to normal physiological saturation ranges for safety reasons.

Trial	Condition	VitalsPatch (SpO₂)	Reference (SpO₂)	Error (SpO₂)
1	Light skin tone	100%	100%	0%
2	darker skin tone	99%	100%	-1%

Table 17. SpO₂ accuracy results

Temperature Accuracy

Temperature measurements were obtained from the MAX30102 internal temperature sensor, corrected using the Phase 3 calibration offset, and compared against handheld thermometer reference measurements. Calibration improved agreement between measured values and reference temperatures. Alert thresholds for elevated and low temperature conditions were verified through controlled testing.

Trial	Condition	VitalsPatch (°F)	Reference (°F)	Error (°F)
1	Fingertip contact (Light skin tone)	85.4	86.2	-0.8
2	Fingertip contact (Darker skin tone)	87.5	87.0	+0.5

Table 18. Temperature accuracy results

Fall Detection

Fall-detection performance was evaluated using controlled motion sequences designed to simulate fall events and normal daily movement. Simulated falls were performed to trigger impact, orientation change, and stillness conditions, while normal motion trials were used to assess false-positive performance.

Trial	Test Type	Trials	Detections	Rate
1	Simulated Fall	10	10	100%
2	Normal Motion (False Positive)	10	0	0%

Table 19. Fall detection results

End-to-End Latency and Dashboard Verification

End-to-end latency testing confirmed that the complete monitoring pipeline operated within the project's target response time. The primary source of delay was the 20-second vital-sign aggregation window, during which three samples were collected at approximately 6.7-second intervals. Additional latency introduced by BLE communication, STM32 forwarding, and dashboard visualization was negligible relative to the acquisition window.

Dashboard verification testing confirmed that heart-rate, SpO₂, temperature, and fall-detection alerts were correctly displayed when corresponding alert conditions were triggered. Dual-patient monitoring functionality also operated as intended, with patient data remaining correctly associated with its originating sensor node.

Impact and Consequences

The VitalsPatch system was designed to improve patient monitoring by providing continuous tracking of heart rate, SpO₂, temperature, and fall events through a single wearable device. In healthcare environments such as nursing homes, rehabilitation centers, and assisted-living facilities, this could allow healthcare providers to monitor multiple patients at once and respond

more quickly when abnormal conditions occur. By combining multiple sensing functions into one device, the system could also reduce the need for several separate monitoring devices.

The primary environmental impact of the project comes from the use of electronic components, batteries, and wearable hardware that may eventually become electronic waste. This impact could be reduced by using rechargeable batteries, designing hardware that can be repaired instead of replaced, and selecting components with longer operational lifetimes. The project also raises ethical considerations related to patient privacy and data security. Although the prototype stores data locally and was developed only for academic purposes, a production version would require encrypted communication, secure data storage, and user authentication on the healthcare providers' side to protect patient information. Another ethical concern is the possibility of false alerts or missed alerts, which could affect patient care if healthcare providers rely on the system for monitoring.

Safety was an important consideration throughout the design process. The wearable electronics operate from a low-voltage battery supply and contain no direct connection to main power, reducing electrical risk to patients. However, inaccurate measurements, missed alerts, or hardware failures could still create safety concerns. These risks could be reduced through additional sensor validation, more extensive clinical testing, waterproof packaging, and redundant fault-detection mechanisms. Many of the environmental, ethical, and safety concerns identified in this project can be reduced through more mindful design decisions early in development. Prioritizing secure data handling, validating measurements against clinical standards, selecting reusable hardware, and designing for reliability would improve both the safety and long-term practicality of the system.

Conclusions and Recommendations

The goal of this project was to determine whether a low-cost wearable system could continuously monitor multiple physiological signals, detect abnormal conditions, and provide centralized monitoring for multiple patients at the same time. Overall, the project showed that this is possible using commercially available components and a distributed embedded architecture. The final system successfully collected heart rate, SpO₂, temperature, accelerometer, and gyroscope data, generated alert codes for abnormal conditions and fall events, and displayed multiple patient information on a centralized dashboard. Testing also showed heart-rate measurements within +/- 4 BPM of reference devices and successful simultaneous monitoring of two patients.

One of the biggest findings from this project was that distributed processing was a better architecture than the original centralized design. Initially, all alert generation and processing occurred on the STM32, but as the system grew it became clear that this would limit scalability. This was also heightened by the STM32 breaking down midway through the full-system integration, making the data processing shift to the ESP32C3 more beneficial in the short and long term. Moving alert generation and fall detection onto the ESP32-C3 nodes simplified the

STM32's role and made it easier to support additional patients in the future. Another important takeaway was that system integration and communication debugging required significantly more development time than sensor acquisition itself. While collecting physiological data was relatively straightforward, building a reliable, accurate end-to-end monitoring pipeline proved to be the more challenging engineering problem.

Although the core monitoring functionality was completed, several limitations remain. The wearable packaging was not completed within the project timeline, temperature measurements relied on a calibrated onboard sensor rather than a dedicated body-temperature sensor, and patient-data security was not implemented beyond local storage. Future work should focus on completing the wearable form factor, expanding the system beyond two patients, improving communication robustness, and implementing secure data handling and storage. Overall, this project demonstrated that a low-cost multi-patient monitoring system is technically feasible and established a solid foundation for future development.

References

- [1]J. Jain, "MAX30102 Pulse Oximeter: SpO2 and Heart Rate Arduino - Zbotic," *Zbotic*, Mar. 11, 2026. <https://zbotic.in/max30102-pulse-oximeter-spo2-and-heart-rate-arduino/> (accessed Jun. 09, 2026).
- [2]"How Apple Watch estimates VO2Max to 1.2 ml/kg/min without a treadmill test," *Empirical Health*, Oct. 22, 2024. <https://www.empirical.health/blog/apple-watch-cardio-fitness-accuracy-vo2max/>
- [3]Turner Medical, "Setting Pulse Oximeter Alarm Levels: A Comprehensive Guide - Turner Medical," *Turnermedical.com*, 2025. <https://www.turnermedical.com/Setting-Pulse-Oximeter-Alarm-Levels-s/178.htm> (accessed Jun. 09, 2026).
- [4]Mayo Clinic, "Hypothermia - Symptoms and Causes," *Mayo Clinic*, Apr. 16, 2024. <https://www.mayoclinic.org/diseases-conditions/hypothermia/symptoms-causes/syc-20352682>
- [5]D. Brajkovic, M. B. Ducharme, and J. Frim, "Relationship between body heat content and finger temperature during cold exposure," *Journal of Applied Physiology*, vol. 90, no. 6, pp. 2445–2452, Jun. 2001, doi: <https://doi.org/10.1152/jappl.2001.90.6.2445>.
- [6]"IEEE Standards Association," *IEEE Standards Association*. <https://standards.ieee.org/ieee/802.15.1/3513/>
- [7]"IEC 60601-1:2024 SER," *Webstore.iec.ch*, 2024. <https://webstore.iec.ch/en/publication/2603>
- [8]ISO - International Organization for Standardization, "ISO 80601-2-61:2017," *ISO*, 2017. <https://www.iso.org/standard/67963.html>
- [9]"MAX30102." Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf>
- [10]"MPU-6500 Product Specification Revision 1.1," 2014. Available: <https://datasheet.octopart.com/MPU-6500-InvenSense-datasheet-138896167.pdf>
- [11]"ESP32C3 Series Datasheet UltraLowPower SoC with RISCV SingleCore CPU Supporting IEEE 802.11b/g/n (2.4 GHz WiFi) and Bluetooth ® 5 (LE) Including." Available: https://files.seeedstudio.com/wiki/XIAO_WiFi/Resources/esp32-c3_datasheet.pdf
- [12]"HM-18/HM-19 CC2640R2 Bluetooth Module Datasheet." Accessed: Jun. 09, 2026. [Online]. Available: https://www.whizzbizz.com/p-httpd/multimedia/HM_18_HM_19_en_V1.pdf
- [13]"STM32F405xx STM32F407xx." Available: <https://www.st.com/resource/en/datasheet/dm00037051.pdf>

[14]“DATASHEET Raspberry Pi 4 Model B Release 1.1,” 2024. Available: <https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008341-DS-1-raspberry-pi-4-datasheet.pdf>

[15]*Arduino.cc*, 2025. <https://docs.arduino.cc/libraries/sparkfun-max3010x-pulse-and-proximity-sensor-library/>

Acknowledgements and Intellectual Property

The VitalsPatch project was developed as part of the University of Washington EE 475 Embedded Systems Capstone course. The system was built using commercially available hardware components and open source software libraries. To the best of the authors' knowledge, the final design does not intentionally infringe upon any existing patents, trademarks, or copyrighted works.

No patents, provisional patents, trademarks, copyrights, trade secrets, or licensing agreements were pursued as part of this project. Although the project resulted in original firmware, software, documentation, and system architecture developed by the team, all work was completed as an academic prototype for educational purposes and no intellectual property protection has been sought at this time.